

Clase 135 — Ofuscación y técnicas anti-reversing

Parte: 5 — Explotación de sistemas y binarios · Fuente: Eilam, Reversing · Practical Binary Analysis 🕒

Duración estimada: 120 min · Nivel: Avanzado

Objetivo

Entender las técnicas que dificultan el reversing —**packing**, cifrado de cadenas, **anti-debugging**, detección de VM, aplanamiento de flujo (control-flow flattening), virtualización de código— y aprender a **derrotarlas** con desempaquetado, parcheo y análisis dinámico. Este conocimiento sirve tanto para analizar malware (Parte 6) como para proteger tu propio software.

⚠️ **Ética:** analiza solo binarios propios/autorizados o muestras en laboratorio aislado.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Clasificar** técnicas de ofuscación y anti-reversing.
- 2. **Detectar** packing por entropía, secciones e imports.
- 3. **Desempaquetar** un binario (UPX y unpacking genérico con dump en runtime).
- 4. **Neutralizar** chequeos anti-debugging (`ptrace` , timing) con parcheo.
- 5. **Reconocer** control-flow flattening y virtualización.

Temas

#	Tema	Por qué importa
1	Packing y cifrado	Ocultan el código real
2	Entropía como indicador	Detección rápida
3	Anti-debugging (ptrace)	Impide adjuntar debugger
4	Anti-VM / anti-sandbox	Evade análisis automático
5	Cadenas cifradas	Nada útil en <code>strings</code>
6	Control-flow flattening	CFG ilegible
7	Virtualización de código	El nivel más duro
8	Contramedidas	Dump, parcheo, dinámico

Definiciones y características

- **Packing:** comprime/cifra el binario y lo descomprime en memoria al ejecutar. *Clave:* el estático solo ve el stub; hay que dumper tras el unpacking.
- **Entropía:** medida de aleatoriedad; secciones empacadas/cifradas tienen entropía alta (~8). *Clave:* primer indicador de ofuscación.
- **Anti-debugging:** detecta debuggers (p. ej. `ptrace(PTRACE_TRACEME)` que falla si ya hay uno). *Clave:* se neutraliza parcheando el chequeo o con `LD_PRELOAD`.
- **Anti-VM:** busca artefactos de virtualización (drivers, MAC, timing). *Clave:* frecuente en malware para evadir sandboxes.
- **Control-flow flattening:** transforma el CFG en un gran switch/dispatcher. *Clave:* rompe la linealidad; se ataca con análisis simbólico/dinámico.
- **Virtualización:** el código se traduce a un bytecode propio interpretado por una VM embebida. *Clave:* la defensa más costosa de revertir.

Herramientas y preparación

```
sudo apt install -y upx binutils
pip install angr          # ejecución simbólica (deofuscación)
# ent o un script python para entropía
```

Trabaja en **VM aislada** con snapshots si manipulas muestras reales.

Laboratorio guiado

Entorno propio / VM aislada.

1. Detecta packing por entropía y secciones:

```
bash upx -t crackme.packed 2>/dev/null && echo "empacado con UPX" python3 - <<'PY' import
math,sys d=open("crackme.packed","rb").read() from collections import Counter c=Counter(d);
H=-sum(n/len(d)*math.log2(n/len(d)) for n in c.values()) print("entropia:",round(H,2)) # ~7.9
sugiere empaquetado/cifrado PY
```

2. Desempaqueta UPX y verifica que ahora ves el código real:

```
bash upx -d crackme.packed -o crackme.unpacked objdump -d crackme.unpacked | head
```

3. Neutraliza anti-debugging basado en `ptrace`: identifica el chequeo en Ghidra y **parchea** el salto (invierte `je` ↔ `jne`) o intercepta `ptrace` con Frida devolviendo 0.
4. Descifra cadenas en runtime: pon un breakpoint tras la rutina de descifrado y dumpea memoria (`dump memory strings.bin $addr $addr+0x200`).
5. Enfréntate a control-flow flattening con angr (ejecución simbólica) para hallar la entrada que llega al estado "éxito":

```
python import angr p = angr.Project("crackme.unpacked", auto_load_libs=False) sm =  
p.factory.simulation_manager(p.factory.entry_state()) sm.explore(find=lambda s: b"Correcto" in  
s.posix.dumps(1)) print(sm.found[0].posix.dumps(0)) # entrada que produce el éxito
```

6. Documenta qué técnica encontraste y cómo la venciste.

Ejercicios

1. Calcula la entropía de un binario normal y de uno empacado; compara.
2. Desempaqueta un UPX y confirma con `objdump` que ves más funciones.
3. Parchea un chequeo `ptrace` para permitir el debugging.
4. Intercepta el anti-debug con Frida sin modificar el binario.
5. Resuelve un crackme con angr y explica sus límites (state explosion).
6. Identifica el dispatcher de un binario con control-flow flattening.

Reto verificable

Toma un `crackme` empacado con anti-debugging y consigue analizarlo: desempácalo, neutraliza el anti-debug y deduce la clave.

Criterio de aceptación: logras adjuntar un debugger pese al anti-debug (parcheo o Frida) y obtienes la clave válida del binario desempacado.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El programa muere al adjuntar GDB	Anti-debug con <code>ptrace</code> ; parchea o hookeo
<code>upx -d</code> falla	No es UPX o está modificado; usa dump en runtime
angr no termina	State explosion; acota con hooks/constraints
Entropía alta pero no es packing	Puede ser datos comprimidos legítimos; verifica secciones
Parcheo rompe el binario	Cambiaste tamaño; parchea solo bytes de opcode equivalentes

Preguntas frecuentes

 **¿Se puede revertir cualquier ofuscación?** En teoría sí con esfuerzo suficiente; la virtualización puede requerir semanas.

 **¿angr es una bala de plata?** No: sufre explosión de estados; úsalo con acotaciones y para funciones concretas.

 **¿Ofuscar mi software lo hace seguro?** Aumenta el coste del atacante, pero no sustituye a una buena arquitectura de seguridad.

Referencias

- Eilam, E. *Reversing: Secrets of Reverse Engineering*. Wiley.

- Andriesse, D. *Practical Binary Analysis*, cap. 12-13. No Starch Press.
- angr — <https://angr.io/>
- Ollydbg/UPX y técnicas anti-debug (Peter Ferrie, "The Ultimate Anti-Debugging Reference").



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

Clase 136 - Fuzzing con AFL++ y libFuzzer